PROYECTO INTEGRADOR – GRUPO 1

Galería de Fotografías con Lista Doblemente Enlazada

1. Datos

Integrantes:	Héctor Cuichan Odalys Chávez Morrison Quillupangui	**Asignatura:** Estructura de Datos **NRC:** 36864

2. Problema Real

Cuando una persona almacena una gran cantidad de fotografías digitales, surge la necesidad indispensable de organizarlas, consultar sus metadatos (título, fecha, ubicación, tamaño), desplazarse eficientemente entre ellas y eliminar aquellas que ya no se desean conservar. Una colección manejada únicamente mediante arreglos estáticos o variables independientes resulta rígida y compleja de modificar.

Para solucionar esta problemática, el presente proyecto propone una galería de fotos administrada mediante una aplicación gráfica en C++ utilizando la biblioteca wxWidgets. Cada fotografía almacena su título, formato, fecha de captura, ruta del archivo y espacio ocupado en megabytes (MB). La aplicación permite gestionar todos estos registros dinámicamente mediante una lista doblemente enlazada y navegar bidireccionalmente a través de la colección visualizando las imágenes en tiempo real.

3. Objetivo

Diseñar e implementar en lenguaje C++ una aplicación de interfaz gráfica (GUI) que administre una galería de fotografías mediante una lista doblemente enlazada genérica. Se aplicarán conceptos avanzados de Tipos de Datos Abstractos (TDA), Clases, Templates, gestión manual de memoria dinámica y una interfaz gráfica e interactiva desarrollada en wxWidgets que permita observar y manipular en tiempo real el catálogo de imágenes y la estructura interna de datos.

4. Justificación de la Estructura

Se seleccionó una lista doblemente enlazada debido a que permite agregar y retirar elementos de manera completamente dinámica durante la ejecución del programa, sin requerir un tamaño fijo predefinido ni reasignaciones masivas de memoria.

Cada nodo de la lista mantiene dos punteros de enlace: uno dirigido hacia el nodo siguiente (siguiente) y otro hacia el nodo anterior (anterior). Gracias a esta bidireccionalidad, la galería puede recorrerse fluidamente en ambos sentidos (adelante y atrás), lo cual constituye la característica funcional perfecta para simular el comportamiento de navegación en visores de imágenes reales.

Además, esta estructura optimiza la inserción y eliminación en posiciones intermedias una vez localizado el nodo de interés, ya que únicamente se requiere la reconexión local de punteros. Como contraparte o limitación estructural, no ofrece acceso aleatorio directo por índice como los arreglos; para acceder a una posición específica es necesario realizar un recorrido secuencial nodo a nodo. Asimismo, cada nodo incurre en un consumo adicional de memoria para almacenar sus dos punteros de control.

5. Diseño del TDA

El TDA principal del sistema es ListaDoble<T>, implementado mediante una plantilla genérica de clase (template). Sus atributos privados centrales son los punteros de control cabeza (cabeza) y cola (cola). Cada nodo interno contiene un dato de tipo T, un puntero siguiente y un puntero anterior. Para esta aplicación específica, el tipo de dato concretado para T es la clase Foto.

5.1. Tipo de Dato Foto

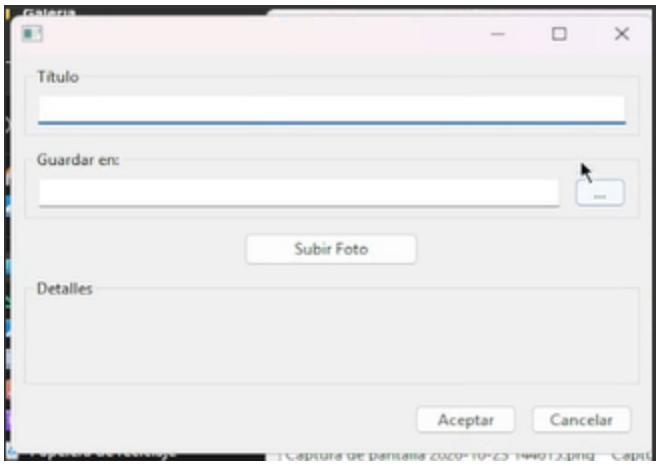
- titulo (string): Título descriptivo de la fotografía.
- formato (string): Extensión o formato gráfico del archivo (ej. JPG, PNG, WEBP).
- fecha (Fecha): Estructura Fecha compuesta por día, mes y año.
- ubicacion (string): Ruta lógica o física del archivo en el sistema de almacenamiento.
- espacio (float): Tamaño del archivo expresado en Megabytes (MB).

5.2. Operaciones de la Lista

Operación	**Descripción**
Constructor	Inicializa los punteros de control cabeza y cola en nullptr.
Destructor	Recorre secuencialmente la lista y libera la memoria reservada dinámicamente para todos los nodos existentes.
enlistarCabeza	Crea un nuevo nodo dinámico y lo enlaza adecuadamente antes de la cabeza actual, actualizando los punteros.
enlistarCola	Crea un nuevo nodo y lo enlaza después de la cola actual, actualizando los punteros de fin de lista.
eliminarNodo	Desconecta el nodo objetivo ajustando los enlaces de sus nodos vecinos (anterior y siguiente) y libera su memoria con delete.
mostrarElemento	Retorna la representación en texto formateado de los metadatos de la foto asociada al nodo recibido.

6. Funcionalidades de la Aplicación

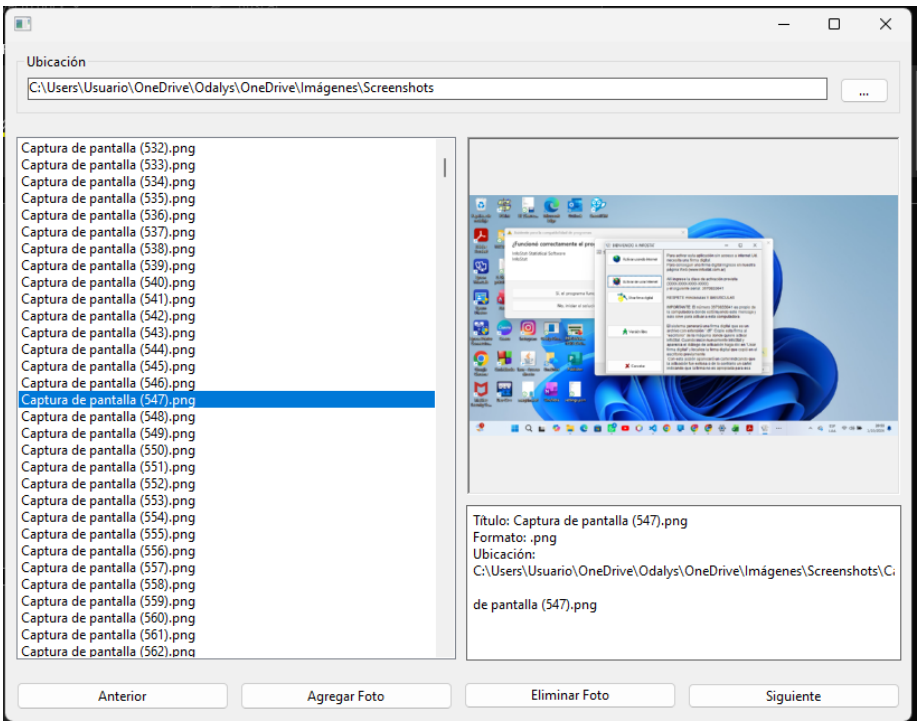
- Seleccionar un directorio local mediante un diálogo dinámico (wxDirDialog) y escanear imágenes compatibles (.jpg, .jpeg, .png, .bmp).
- Cargar automáticamente todas las imágenes del directorio en la estructura de datos ListaDoble y refrescar el visor de lista (wxListBox).
- Visualizar la imagen seleccionada adaptada proporcionalmente en un contenedor wxStaticBitmap manteniendo su relación de aspecto.
- Desplegar en tiempo real los metadatos completos de la foto seleccionada (título, formato, ubicación, fecha y espacio en MB).
- Navegación bidireccional mediante botones "Anterior" y "Siguiente", recorriendo los nodos a través de los punteros anterior y siguiente.
- Agregar nuevas imágenes mediante el cuadro de diálogo gráfico AgregaFoto, permitiendo elegir archivo de origen, título personalizado y carpeta destino.
- Eliminar imágenes tanto de la lista en memoria como del disco mediante confirmación de usuario, manteniendo sincronizada la estructura ListaDoble.



7. Interfaz de Usuario (GUI con wxWidgets)

La aplicación cuenta con una interfaz gráfica completa desarrollada con la biblioteca wxWidgets y diseñada mediante wxSmith en Code::Blocks. La ventana principal (GaleriaFrame) se organiza visualmente en:

- **Barra Superior de Ubicación:** Muestra la ruta del directorio activo e incluye el botón "..." para explorarlo.
- **Panel Izquierdo:** Un componente ListBox1 que muestra el listado de archivos de imágenes escaneados.
- **Panel Derecho:** Un visor wxStaticBitmap que renderiza la imagen reescalada con alta calidad (wxIMAGE_QUALITY_HIGH) y un cuadro de texto wxStaticText que muestra sus metadatos.
- **Barra Inferior de Controles:** Botones interactivos para navegar ("Anterior", "Siguiente"), agregar una nueva imagen ("Agregar Foto") y remover la foto seleccionada ("Eliminar Foto")



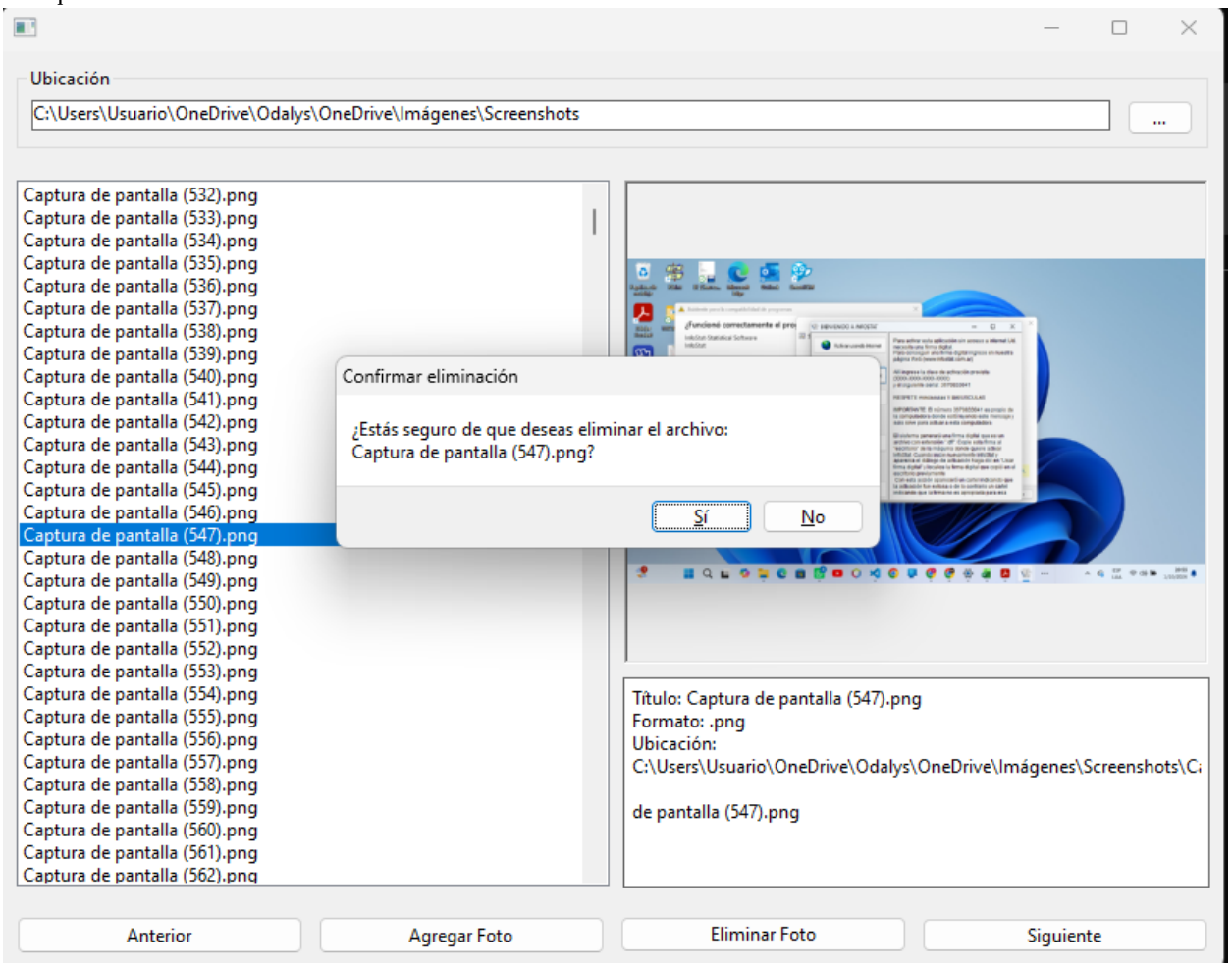
8. Gestión de Memoria y Casos Límite

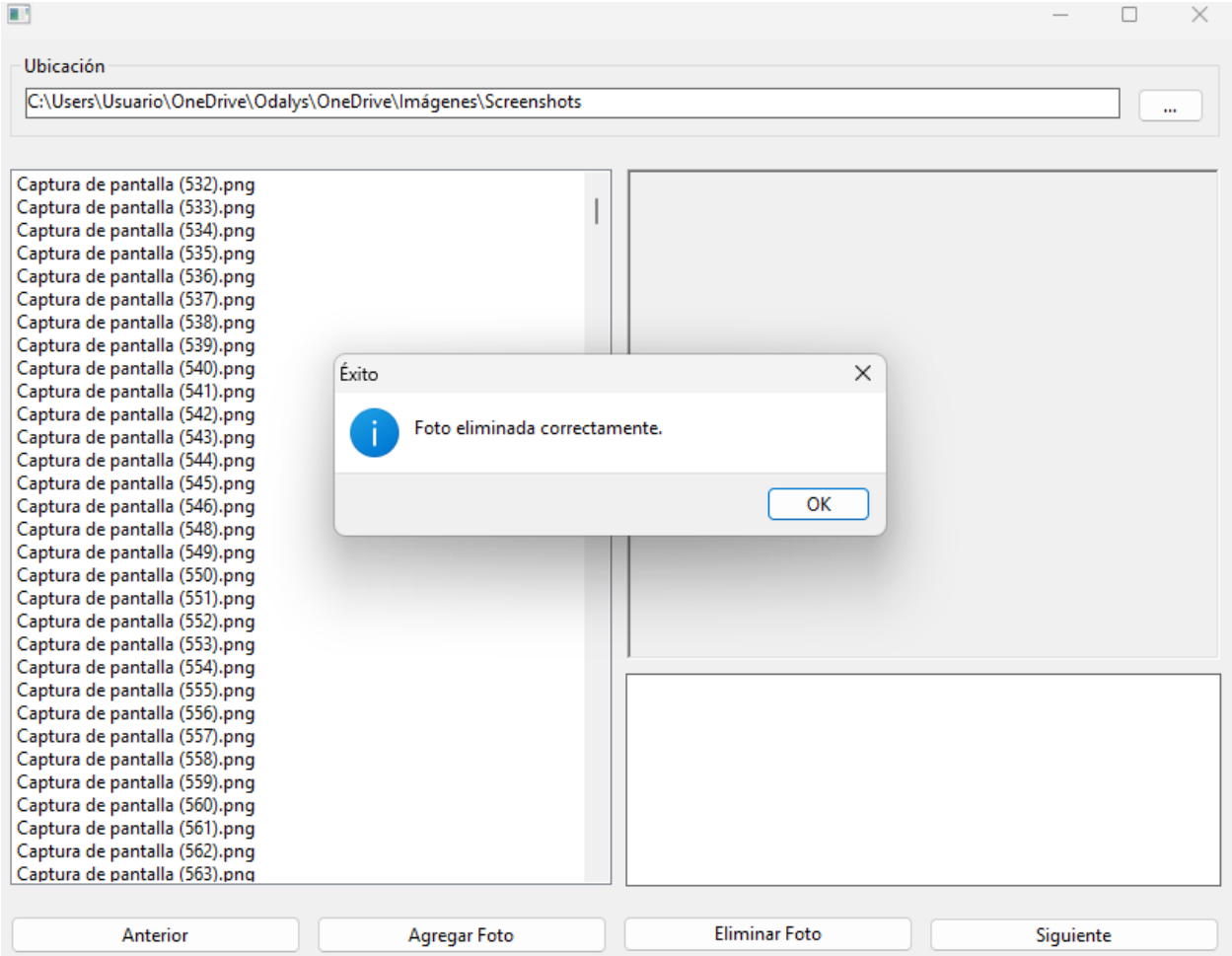
La memoria para cada nodo se reserva dinámicamente en el Heap mediante el operador new. Durante un proceso de eliminación, la estructura desvincula primero los enlaces de los nodos contiguos para preservar la integridad de la lista y posteriormente libera el bloque de memoria objetivo con el operador delete.

Para prevenir desbordamientos o fugas de memoria (memory leaks), el destructor de la clase recorre secuencialmente la lista desde la cabeza liberando uno a uno todos los nodos. Adicionalmente, se deshabilitan o privatizan el constructor de copia y el operador de asignación para evitar copias superficiales (shallow copy) inadvertidas de los punteros.

Caso límite y validaciones contempladas:

- **Lista vacía:** Despliegue de advertencia mediante wxMessageBox e inhibición de operaciones sobre punteros nulos.
- **Elemento único:** Al eliminar el único nodo presente, tanto cabeza como cola se reasignan a nullptr.
- **Eliminar el primer nodo (cabeza):** La nueva cabeza actualiza su puntero anterior a nullptr.
- **Eliminar el último nodo (cola):** La nueva cola actualiza su puntero siguiente a nullptr.
- **Sincronización de interfaz:** Desactivación o reinicio del visor de imágenes y metadatos cuando la lista queda vacía.





9. Organización de Archivos

Archivo	**Responsabilidad y Contenido**
Foto.h	Define la estructura Fecha y la clase Foto con sus atributos, getters, setters y el método mostrarDatos().
ListaDobleEnlazada.h	Declara la plantilla genérica ListaDoble<T>, la estructura del Nodo y sus prototipos (enlistarCola, eliminarNodo, etc.).
Operaciones_ListaDoble.h	Implementa los métodos dinámicos de ListaDoble<T> y la gestión manual de memoria (new/delete).
GaleriaApp.h	Declara la clase principal de la aplicación wxWidgets (GaleriaApp).
GaleriaApp.cpp	Implementa la inicialización de la aplicación gráfica OnInit() con wxInitAllImageHandlers().
GaleriaMain.h	Declara la ventana principal GaleriaFrame, componentes visuales, eventos y la lista doble de fotografías.
GaleriaMain.cpp	Implementa la lógica de la ventana principal: carga de directorio, pintado de fotos, eliminación y navegación.
AgregaFoto.h	Declara la ventana secundaria AgregaFoto para el registro y copia de nuevas imágenes.

AgregaFoto.cpp	Implementa la selección de archivos de origen/destino, copia en disco (wxCopyFile) e inserción en ListaDoble.
resource.rc / Galeria.cbp	Archivo de recursos de Windows y proyecto oficial de Code::Blocks con configuración de compilación.

10. Librerías Seleccionadas

Biblioteca / Header	**Uso y Propósito en el Proyecto**
<wx/app.h>	Definición y control del ciclo de vida de la aplicación GUI en wxWidgets.
<wx/frame.h>, <wx/panel.h>	Construcción de ventanas y paneles contenedores de la interfaz gráfica.
<wx/statbmp.h>, <wx/image.h>	Carga, manipulación, procesamiento de píxeles y escalado de imágenes gráficas.
<wx/listbox.h>	Componente de lista para mostrar y seleccionar nombres de archivos en la interfaz GUI.
<wx/dirdlg.h>, <wx/filedlg.h>	Cuadros de diálogo del sistema operativo para explorar carpetas y seleccionar archivos.
<wx/msgdlg.h>	Despliegue de mensajes emergentes de información, error o confirmaciones.
<wx/filename.h>, <wx/filefn.h>	Operaciones con el sistema de archivos (obtener extensiones, tamaños, wxCopyFile, wxRemoveFile).

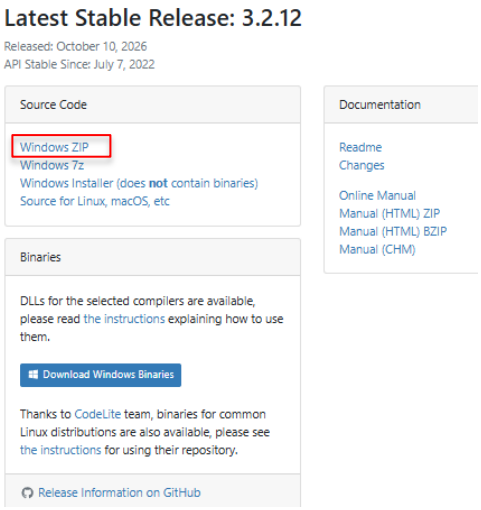
11. Instalación y Configuración de TDM-GCC y wxWidgets

11.1. Instalación de TDM-GCC

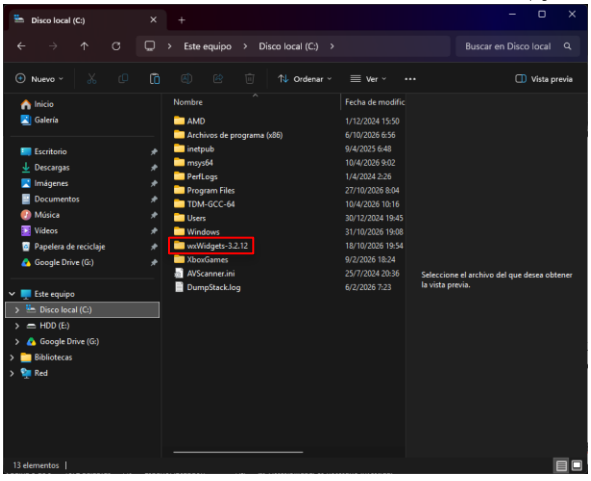
- Ingresar a la página oficial: https://jmeubank.github.io/tdm-gcc/download/
- Descargar el instalador de TDM-GCC para Windows de 64 bits.
- Ejecutar el instalador y aceptar los permisos.
- Seleccionar la opción de instalación correspondiente a C++.
- Elegir la carpeta de instalación y completar el proceso.

11.2. Descarga de wxWidgets

- Ingresar a la página oficial: https://www.wxwidgets.org/downloads/
- Descargar el paquete comprimido de wxWidgets 3.2.x.



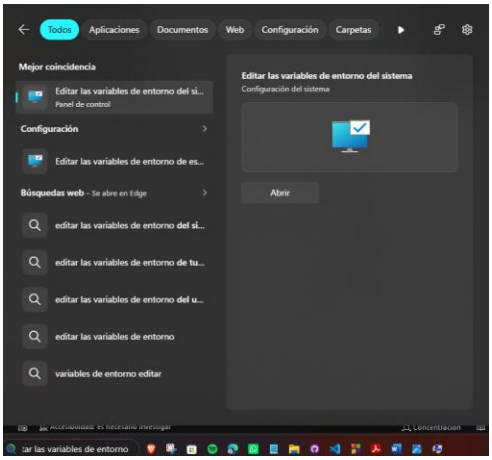
- Extraer el contenido en la raíz del disco local C: (ej. C:\wxWidgets-3.2.12).



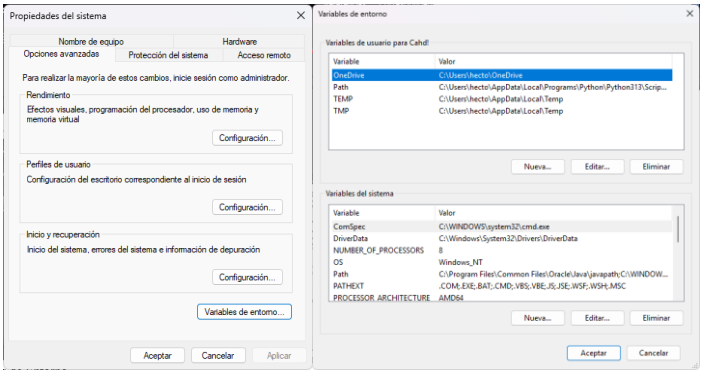
- Verificar que la estructura de carpetas contenga el directorio build\msw

11.3. Configuración de las Variables de Entorno

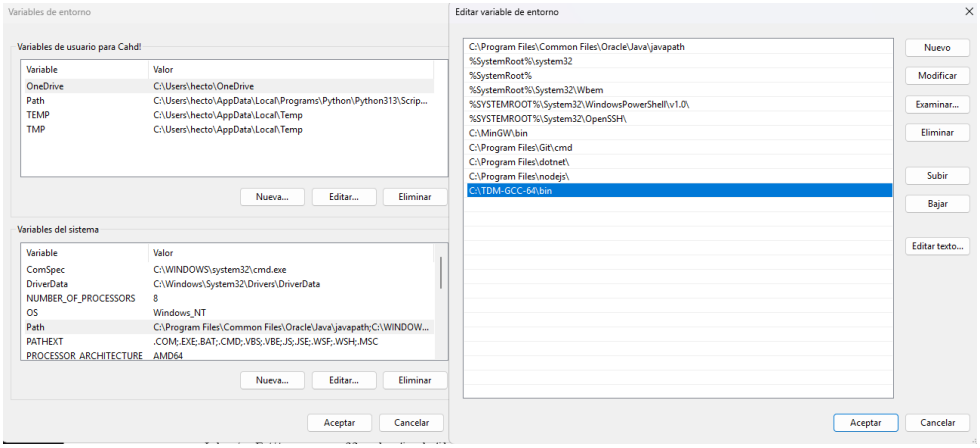
- Abrir las Opciones Avanzadas del Sistema en Windows.



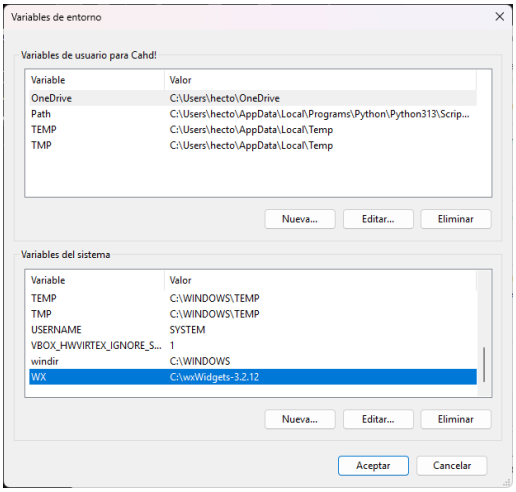
- Ingresar al apartado de Variables de Entorno.



- En la variable de sistema Path, agregar la ruta bin de TDM-GCC (ej. C:\TDM-GCC-64\bin).



- Crear la nueva variable de entorno WX apuntando a la ruta de wxWidgets (ej. C:\wxWidgets-3.2.12).



11.4. Compilación de wxWidgets

Abrir la consola de comandos (cmd) y navegar hasta la carpeta build\msw de wxWidgets:

Comandos:

- Librerías Estáticas: mingw32-make -f makefile.gcc RUNTIME_LIBS=static USE_XRC=1MONOLITHIC=1 SHARED=0 UNICODE=1 BUILD=release TARGET_CPU=X64 CXXFLAGS="-std=gnu++11"
- Librerías Dinámicas: mingw32-make -f makefile.gcc RUNTIME_LIBS=dynamic USE_XRC=1 MONOLITHIC=1 SHARED=1 UNICODE=1 BUILD=release TARGET_CPU=X64 CXXFLAGS="-std=gnu++11"

11.5. Verificación

Una vez terminada la instalación y configuración, se verifica que TDM-GCC y wxWidgets estén correctamente instalados y que las variables de entorno tengan las rutas correspondientes.

- Video tutorial de referencia: https://www.youtube.com/watch?v=v_v9HtaoPRs

11.6. Generación del Archivo Ejecutable (.exe)

En Code::Blocks, tras compilar en modo Release o Debug, el ejecutable autónomo se genera en la carpeta bin\Debug\Galeria.exe listo para ejecutarse directamente en cualquier sistema Windows sin necesidad del IDE.

11.7. Guía de Ejecución del Programa

- Abrir la aplicación haciendo doble clic en Galeria.exe.
- Hacer clic en el botón "..." para seleccionar una carpeta con imágenes (.jpg, .png, .bmp).
- Seleccionar cualquier foto de la lista para visualizarla y consultar sus metadatos.
- Utilizar los botones "Anterior" y "Siguiente" para desplazarse bidireccionalmente por la lista doble.
- Usar el botón "Agregar Foto" para subir y guardar una nueva imagen, o "Eliminar Foto" para retirarla.